# Computer organization and Architecture

Lecturer Engr. Sana Fatima

# Marks Distribution

- Mids (25%)
- Finals (50)
- Sessional (25 %)
- Attendance (10%)
- Assignments (4 assignments), Quizes (4 quizes) (15%)

# Table of Contents

| Module / Unit | Topics Covered |
|---|---|
| **Unit 1: Introduction & Digital Logic Review** | - Structure of computer systems, architecture vs organization<br>- - Number systems, data formats, error detection<br>- - Boolean algebra, logic gates, minimization (K-maps)<br>- - Combinational circuits: multiplexers, decoders, adders<br>- - Sequential circuits: latches, flip-flops, registers, counters |
| **Unit 2: Digital Components & Basic Circuits** | - Integrated circuits, families<br>- - Registers, shift registers<br>- - Memory elements<br>- - Buses and data paths<br>- - Basic ALU circuits |
| **Unit 3: Data Representation & Arithmetic** | - Signed and unsigned numbers , - 2's complement<br>- - Floating-point representation<br>- - Arithmetic operations: addition, subtraction, multiplication, division - Arithmetic logic unit (ALU) design |
| **Unit 4: Register Transfer, Micro-operations** | - Register Transfer Language (RTL)<br>- - Micro-operations: arithmetic, logic, shift<br>- - Bus and memory transfers<br>- - Control signals and timing |
| **Unit 5: Basic Computer Organization & Design** | - Instruction codes, computer registers<br>- - Instruction set architecture, addressing modes<br>- - Instruction cycle (fetch, decode, execute)<br>- - Timing and control - Designing a "Basic Computer" (accumulator based) |
| **Unit 6: Programming the Basic Computer / Assembly** | - Machine language & assembly language<br>- - Assembler - Subroutines, loops - Input/output programming, interrupts |

# Table of Contents

| Module / Unit | Topics Covered |
|---|---|
| **Unit 7: Control Unit Design** | - Hardwired control - Microprogrammed (microcode) control<br>- - Control memory, microinstruction sequencing - Examples of control unit design |
| **Unit 8: Memory Organization** | - Hierarchy: registers, cache, main memory, secondary<br>- - Cache organization, mapping, replacement policies<br>- - Virtual memory<br>- - Associative memory, memory management<br>- - Interleaving, error correction |
| **Unit 9: Input / Output Organization & Interrupts** | - I/O techniques: programmed I/O, DMA, interrupt-driven<br>- - I/O modules, controllers, buses - Interrupt structure, priority<br>- - Interface circuits - Example I/O handling in the “basic computer” model |
| **Unit 10: Pipeline, Vector, & Parallel Processing** | - Pipelining principles, hazards, pipeline control - Superscalar, VLIW, instruction-level parallelism - -- Flynn’s classification (SISD, SIMD, MIMD) - ----- Multiprocessors: shared memory, interconnections, synchronization - Cache coherence in multiprocessor systems |

# Objective

- What is Computer architecture?
- What is Computer organization?
- Von Neumann Architecture
- Key Components of Von Neumann Architecture
- Harvard Architecture
- Functional Units & Basic Operation

# Computer Architecture

**Definition**:

Computer Architecture refers to the **functional behavior of a computer system as seen by a programmer**.

It defines *what the system does* the instruction set, addressing modes, data types, and I/O mechanisms.

In simple words: **Architecture = what the computer is designed to do.**

**Examples**:

- The **Instruction Set Architecture (ISA)** of Intel x86 vs. ARM.
- x86 and ARM have different instructions (e.g., MOV AX, BX vs. LDR R0, [R1]).
- This difference is **architecture**, because it's visible to the programmer.
- A system might support **32-bit or 64-bit instructions** → this is architectural.
- The **Von Neumann architecture** concept (same memory for data & instructions).

# Computer Organization

- **Definition**:
  Computer Organization refers to the **operational units and their interconnections that implement the architecture**. It deals with *how the system is built internally* — control signals, hardware components, pipelines, memory hierarchy, etc.
- In simple words: **Organization = how the computer is built to do it.**
- **Examples**:
  - Whether a processor has **one ALU** or **multiple ALUs** working in parallel.
  - Whether instructions are executed in a **single-cycle** or **pipelined** manner.
  - Cache size and levels (L1, L2, L3) → organizational detail.
  - Two different Intel i7 processors (same architecture, x86 ISA) but with **different number of cores, clock speeds, or cache** → same architecture, different organization.

# Computer architecture vs Computer Organization

| Aspect | **Computer Architecture** | **Computer Organization** |
|---|---|---|
| **Meaning** | What the computer does (programmer's view) | How the computer is built internally (hardware view) |
| **Focus** | Instruction Set, addressing modes, data types | Control signals, ALU, memory hierarchy, pipelines |
| **Visible to Programmer?** | Yes | No |
| **Example** | 64-bit instruction set, Von Neumann model | Number of ALUs, pipeline depth, cache design |

# Simple Example to Remember:

**Example:**

- **Computer Architecture** (what you see as a user/programmer):
- The laptop supports a **64-bit instruction set**.
- It has instructions like **ADD, SUB, LOAD, STORE**.
- It can run Windows or Linux.
  ☞ This is the **programmer's view** — the features the system provides.
- **Computer Organization** (how it actually works inside):
  - The laptop has a **quad-core processor** with **pipelining**.
  - It uses **cache memory** (L1, L2) to speed up execution.
  - It fetches instructions and data through **control signals** and buses.
    ☞ This is the **hardware view** — how those features are implemented.

# Easy Rule

- **Architecture** = What instructions/features are available.
- **Organization** = How the hardware is arranged to run those instructions.

# 1. Von Neumann Architecture — History

- Proposed by **John von Neumann** in **1945**, based on the **EDVAC** design.
- The key idea: **Stored Program Concept** → both **instructions** (programs) and **data** are stored in the **same memory**.
- ☞ **Example**:
  When you open Microsoft Word, the **program code (instructions like ADD A, B )** and the **text you type (data)** are both stored in RAM. The CPU fetches both from the same memory.

# Key Components of Von Neumann Architecture

**1.Central Processing Unit (CPU):**

- **ALU (Arithmetic Logic Unit)** → performs operations (add, subtract, AND, OR).
  - Example: If you calculate (5 + 3, ALU performs the addition.)
- **Control Unit (CU)** → directs data flow, tells when to fetch, decode, and execute.
  - Example: CU sends signals to fetch the next instruction from memory.

**2.Single Memory:**

- Stores both instructions (e.g., ADD R1, R2) and data (e.g., value 5).
- Example: In RAM, line 1000 might store an instruction, and line 1001 might store a number.

# Key Components of Von Neumann Architecture

**3. Input / Output Modules:**

- Allows data to enter (keyboard, mouse) and results to leave (monitor, printer).

**4. Buses (shared communication paths):**

- **Data Bus** → transfers instructions/data.
- **Address Bus** → tells where to fetch/store.
- **Control Bus** → signals for timing and control.

# Von Neumann Bottleneck

- Because **one bus** carries both **instructions and data**, the CPU cannot fetch an instruction and access data at the same time.
- This creates a **bandwidth bottleneck**.

**Example:**

If CPU wants to execute ADD A, B, it must:

- ❑ Fetch the ADD instruction from memory.
- ❑ Fetch the values of A and B.

Both share the **same bus**, so it takes more time (slows performance).

# Harvard Architecture (Alternative)

- Uses **separate memories and buses** for **data** and **instructions**.
- CPU can fetch an instruction *while* reading/writing data → **faster**.
- ☞ **Example:**
- **Von Neumann** = one notebook for both **recipe + ingredients list** → you must flip pages back and forth (slower).
- **Harvard** = two notebooks, one for **recipe** and one for **ingredients** → you can read both at the same time (faster).

# Comparison:

- Von Neumann: simpler, cheaper (used in PCs, laptops).
- Harvard: faster, but more complex and costly.

# Functional Units & Basic Operation

## Basic Functional Units

- **ALU:** arithmetic & logic ops (e.g., 5+7=12).
- **Registers**: small, fast storage inside CPU (e.g., R1=5, R2=7).
- **Control Unit**: directs data flow (fetch instruction, decode, execute).
- **Memory**: stores instructions and data.
- **Input/Output**: devices for interaction.

# Instruction Cycle (Step-by-Step Example)

Suppose we want to compute **C = A + B**.

**1.Fetch**:

- CU fetches the ADD A, B instruction from memory.

**2.Decode**:

- CU decodes: operation = ADD, operands = A & B.

**3.Execute**:

- ALU adds values of A and B.

**4.Memory Access**:

- Result stored in register or memory.

**5.Write Back**:

- Result is written back to memory location C.

# Performance & Tradeoffs

- **Speed:**
  - Pipelining (fetch next instruction while executing current) improves speed.
  - Example: Modern Intel CPUs use deep pipelines (up to 14+ stages).
- **Cost:**
  - Von Neumann = cheaper (one memory, one bus).
  - Harvard = costlier (two memories, separate buses).
- **Power Consumption:**
  - More buses/memory = higher power.
  - Embedded systems use Harvard but optimize for low power.
- **Scalability:**
  - Adding more cores, cache, and pipelines makes systems faster.
  - Example: A quad-core i5 (same architecture as i7) differs in **organization** (cache size, cores).

# Reference book

- M. Morris Mano — *Computer System Architecture*
- William Stallings — *Computer Organization and Architecture: Designing for Performance*